iOS Sale Item Category Service Guide

The Merchant Sale Category Service provides all sale category-related operations—such as retrieval, creation, updating, and deletion of sale category data.

Below are the key utility features available:

- **Create Merchant Sale Categories**: Creates one or more sale categories under the merchant's account.
- **Get All/Specific Merchant Sale Categories**: Processes the Get Sale Item Category request and retrieves the corresponding categories for the merchant. This can be used to obtain details for a specific category or to fetch the complete list of categories associated with the requested user.
- **Update Existing Merchant Sale Categories**: Processes the Update Merchant Sale Category request and updates the specified sale categories for the merchant.
- **Delete Merchant Sale Categories**: Processes the Delete Merchant Sale Category request and removes the specified sale category for the merchant.

Prerequisites

Before you continue, ensure you have the following prerequisites completed and ready:

- **Merchant Client Application Credentials and Access Token:** Ensure that merchant onboarding is complete and that you've received your client application credentials and access token before proceeding.
- **Access to Deluxe Developer Portal:** This portal provides additional resources, API documentation, and integration tools.
- **Merchant Authorization & Accounts Guide:** This step is required to obtain the merchant authorization token and

ON THIS PAGE

Prerequisites

Create Merchant Sale Categories

Get All/Specific Merchant Sale Categories

Update Existing Merchant Sale Categories

Delete Merchant Sale Categories

Note: *If you don't have a Deluxe Merchant Account or Access Token, simply visit Merchant Services Get Started and contact us today.*

Create Merchant Sale Categories

Creates one or more sale categories under the merchant's account.

Step 1: Create a SaleItemCategoryService Object

```
let merchantCategoryService =
SaleItemCategoryService()
```

Step 2 : Create a Merchant Sale Category

Construct a CreateSaleItemCategoryRequest by providing the required input parameters, including categoryList, which is a list of SaleItemCategory objects containing details of the categories to be created.

```
let saleItemCategory = SaleItemCategory(
 categoryID: 101,
 categoryName: "Beverages"
)
let request = CreateSaleItemCategoryRequest(
 accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
 categoryList:
[saleItemCategory1,saleItemCategory2]
)
```

accessToken: This should be the access token for the merchant account selected by the user. Available merchant accounts can be retrieved using the MerchantAccountService.

Pass the request and bearer token to the createSaleItemCategories() method.

```
merchantCategoryService.createSaleItemCategories(re
quest: request, bearerToken: bearerToken) {
response, error in
 if let result = response {

 } else if let error = error {
 }
}
```

Step 3: Response Handling

On success, createSaleItemCategories() method returns a CreateSaleItemCategoryResponse object which contains the following:

- **If success = true**, response contains list of SaleItemCategory objects that were successfully created.
- **If success = false**, the response indicates a failure, and the corresponding reasons can be found in the errorCode and message fields.

On failure, the method returns an error object that describes the cause of the failure.

Get All/Specific Merchant Sale Categories

Processes the Get Sale Item Category request and retrieves the corresponding categories for the merchant. This can be used to obtain details for a specific category or to fetch the complete list of categories associated with the requested user.

Step 1: Create a SaleItemCategoryService Object

```
let merchantCategoryService =
SaleItemCategoryService()
```

Step 2: Get Merchant Sale Categories

The SaleItemCategoryGetRequest object is used to define the data required to fetch merchant sale categories, including:

- **accessToken**: This should be the access token for the merchant account selected by the user. Available merchant accounts can be retrieved using the MerchantAccountService.
- **categoryID**: ID of a specific sale item category to retrieve, or null to fetch all.
- **page**: The page number for pagination.
- **pageSize**: The number of items to fetch per page.

```
let request = SaleItemCategoryGetRequest(
 accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
```

Pass the request and bearer token to the getSaleItemCategory() method.

```
merchantCategoryService.getSaleItemCategory(request
: request, bearerToken: bearerToken) { response,
error in
 if let result = response {
 } else if let error = error {
 }
}
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

Step 3: Response Handling

On success, getSaleItemCategory() method returns a GetSaleItemCategoryResponse object which contains the following:

- **If success = true**, response contains list of SaleItemCategory objects that were successfully fetched.
- **If success = false**, the response indicates a failure, and the corresponding reasons can be found in the errorCode and message fields.

On failure, the method returns an error object that describes the cause of the failure.

Update Existing Merchant Sale Categories

Processes the Update Merchant Sale Category request and applies the specified updates to the merchant's sale categories.

Step 1: Create a SaleItemCategoryService Object

```
let merchantCategoryService =
SaleItemCategoryService()
```

Step 2: Update Merchant Sale Category

The SaleItemCategoryUpdateRequest object is used to define the data required to update sale categories for a merchant account, including a categoryList which is a list of

```
let saleItemCategory = SaleItemCategory(
 categoryID: 101,
 categoryName: "Beverages"
)
let updateRequest = SaleItemCategoryUpdateRequest(
 accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
 categoryList: [saleItemCategory1,
saleItemCategory2]
)
```

accessToken: This should be the access token for the merchant account selected by the user. Available merchant accounts can be retrieved using the MerchantAccountService.

Pass the request and bearer token to the updateSaleItemCategories() method.

```
merchantCategoryService.updateSaleItemCategories(re
quest: updateRequest, bearerToken: bearerToken) {
response, error in
 if let result = response {

 } else if let error = error {

 }
}
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

Step 3: Response Handling

On success, the method returns a UpdateSaleItemCategoryResponse object containing the following:

- **If success = true**, response contains a list of SaleItemCategory objects that were successfully updated.
- **If success = false**, the response indicates a failure, and the corresponding reasons can be found in the errorCode and message fields.

On failure, the method returns an error object that describes the cause of the failure.

Delete Merchant Sale Categories

Step 1: Create a SaleItemCategoryService Object

```
let merchantCategoryService =
SaleItemCategoryService()
```

Step 2: Delete a Merchant Sale Category

Construct a SaleItemCategoryDeleteRequest by providing the required input parameters, including a categoryList which is a list of MerchantSaleItemCategory objects representing the sale item categories to be deleted, each containing details such as categoryId and categoryName.

```
let saleItemCategory = SaleItemCategory(
 categoryID: 101,
 categoryName: "Beverages"
)
let deleteRequest = SaleItemCategoryDeleteRequest(
 accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
 categoryList: [saleItemCategory1,
saleItemCategory2]
)
```

accessToken: This should be the access token for the merchant account selected by the user. Available merchant accounts can be retrieved using the MerchantAccountService.

Pass the request and bearer token to the deleteMerchantSaleItems() method.

```
merchantCategoryService.deleteSaleItemCategories(re
quest: deleteRequest, bearerToken: bearerToken) {
response, error in
 if let result = response {

 } else if let error = error {
 }
}
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

Step 3: Response Handling

On success, the method returns a DeleteSaleItemCategoryResponse object containing the following:

corresponding reasons can be found in the errorCode and message fields.

On failure, the method returns an error object that describes the cause of the failure.

Note: *Please refer to the Complete SDK Reference for additional details iOS SDK Reference Guide*

Deluxe.com | Privacy Policy | Do Not Sell My Personal Information

Deluxe Corporation, 801 S Marquette Ave, Minneapolis, MN 55402.

© 2026 Deluxe Corporation